

**Міністерство освіти і науки України**  
**Національний університет "Львівська політехніка"**



**Звіт до лабораторної роботи №10**  
**З дисципліни "Web-технології та Web-дизайн"**  
**на тему: "React.js: Redux: Cart page (shopping cart)"**

**Виконала:**

Ст.гр. ІР-11

Луцан І.З.

**Прийняв:**

доцент, к.т.н.

Іванюк О.О.

## Завдання:

### 10. React.js: Redux: Cart page (shopping cart)

**Description:** You are on your way to finishing this insane project... Create the first of three cart pages - Shopping cart page.

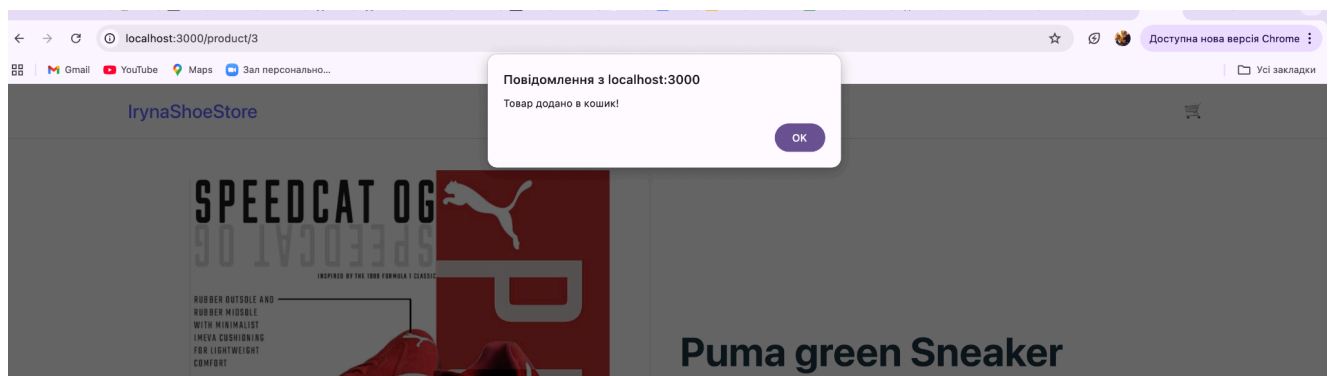
Also, here you meet one of the most popular React library - Redux.

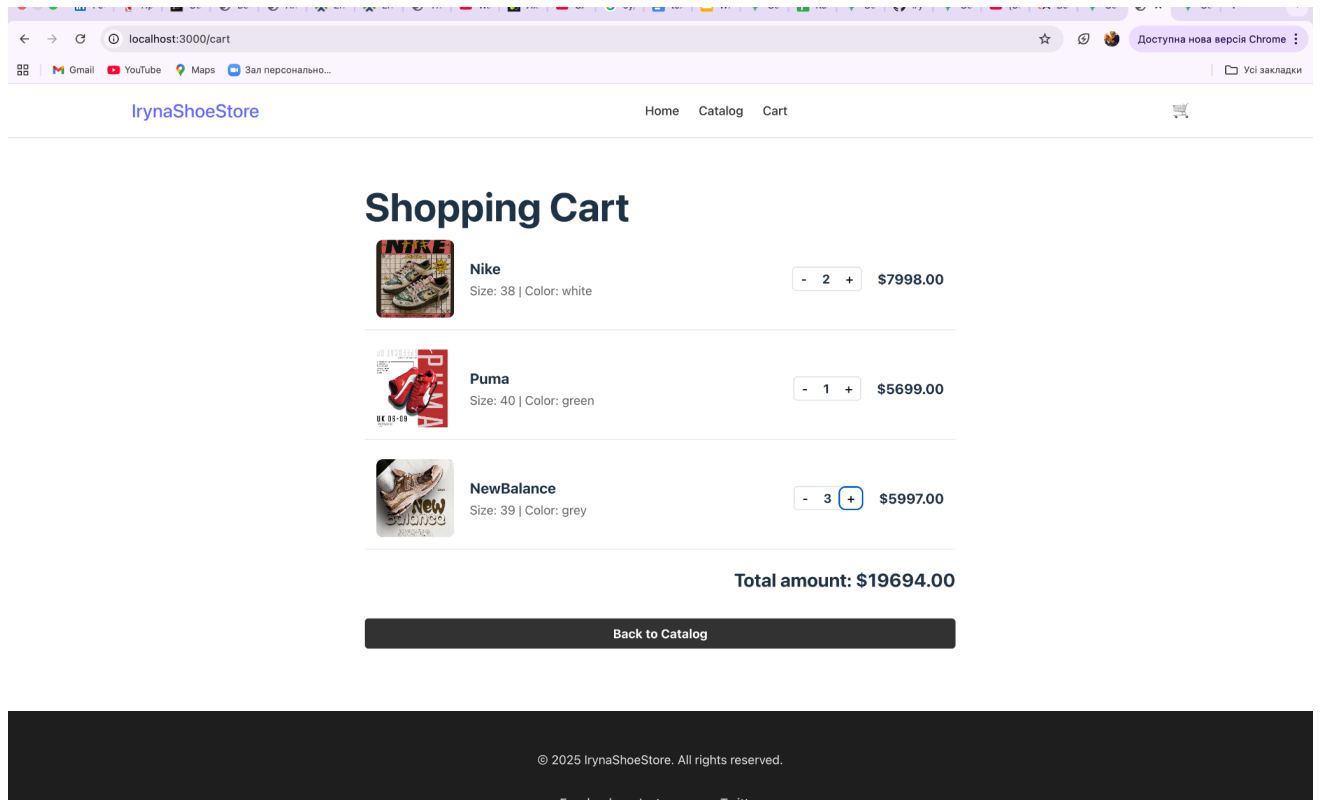
*Variants - (products that you are 'selling') the same as for previous works. (see the description to 3rd work)*

#### Requirements:

- All of the requirements for previous React.js works should be kept.
- **Functionality:**
  - Item page: "Add to cart" action should be implemented using Redux flow: when you add an item to cart, it should be added to your redux store. On Cart page you take all of the items from the store
  - Cart page: "add/remove" actions should be implemented through redux actions & reducers as well.
- **Code style:**
  - Redux: All Redux parts (actions / reducers / store) should be kept in separate and specific files (actions.js / reducers.js / store.js etc.)
  - Use useSelector hook for getting the data from redux store (instead of connect() function)  
<https://react-redux.js.org/api/hooks#useselector-examples>
  - Use useDispatch hook for dispatching your actions (instead of connect() function)  
<https://react-redux.js.org/api/hooks#usedispatch>

## Нові зміни на сайті:





## Файл main.jsx:

```
import { StrictMode } from 'react';
import { createRoot } from 'react-dom/client';
import { BrowserRouter } from 'react-router-dom';
import { Provider } from 'react-redux';
import store from './redux/store';
import './index.css';
import App from './App.jsx';

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <Provider store={store}>    {/* дозволяє даним діяти до будь-якого компонента */}
      <BrowserRouter>
        <App />
      </BrowserRouter>
    </Provider>
  </StrictMode>
);
```

## Файл CardPage.jsx:

```
import React from 'react';
import { useSelector, useDispatch } from 'react-redux';
import { removeFromCart, removeOneFromCart, addToCart } from '../redux/actions';
import PrimaryButton from '../containers/Components/ui/PrimaryButton';
import { Link } from 'react-router-dom';
```

```
import nikeImage from '../assets/Nike.jpg';
import adidasImage from '../assets/Adidas.jpg';
import asicsImage from '../assets/Asics.jpg';
import converseImage from '../assets/Converse.jpg';
import newbalanceImage from '../assets/Newbalance.jpg';
import pumaImage from '../assets/Puma.jpg';
import salomonImage from '../assets/Salomon.jpg';
import defaultShoeImage from '../assets/shoes_store.jpg';

const producerImages = {
  'nike': nikeImage,
  'adidas': adidasImage,
  'asics': asicsImage,
  'converse': converseImage,
  'newbalance': newbalanceImage,
  'puma': pumaImage,
  'salomon': salomonImage,
};

const cartContainerStyles = {
  maxWidth: '800px',
  margin: '40px auto',
  padding: '20px',
};

const cartItemStyles = {
  display: 'flex',
  justifyContent: 'space-between',
  alignItems: 'center',
  padding: '15px',
  borderBottom: '1px solid #eee',
  marginBottom: '10px',
};

const totalStyles = {
  fontSize: '24px',
  fontWeight: 'bold',
  textAlign: 'right',
  marginTop: '20px',
};

function CartPage() {
  // дістаємо товари з Redux-сховища
  const cartItems = useSelector((state) => state.cartItems);
  const dispatch = useDispatch();

  // створюється об'єкт, де ключ це ID, а значенням – товар + лічильник
  const groupedItems = cartItems.reduce((acc, item) => {
    if (acc[item.id]) {
      acc[item.id].count += 1; // якщо товар вже є, збільшуємо кількість
    }
  }, {});
}
```

```

    } else {
      acc[item.id] = { ...item, count: 1 }; // якщо немає, додаємо з кількістю 1
    }
    return acc;
  }, {}));

// перетворюємо об'єкт назад у масив для відображення
const displayItems = Object.values(groupedItems);

const totalAmount = cartItems.reduce((total, item) => total + item.price, 0);

const handleRemove = (id) => {
  dispatch(removeFromCart(id));
};

if (cartItems.length === 0) {
  return (
    <div style={{ textAlign: 'center', padding: '50px' }}>
      <h2>Ваш кошик порожній 🛒</h2>
      <Link to="/catalog">
        <PrimaryButton>Перейти до каталогу</PrimaryButton>
      </Link>
    </div>
  );
}

return (
  <div style={cartContainerStyles}>
    <h1>Shopping Cart</h1>

    <div className="cart-list">
      {displayItems.map((item) => (
        <div key={item.id} style={cartItemStyles}>

          {/* картинка + опис */}
          <div style={{ display: 'flex', alignItems: 'center', gap: '20px' }}>
            <img
              src={producerImages[item.producer.toLowerCase().trim()] || defaultShoeImage}
              alt={item.producer}
              style={{ width: '100px', height: '100px', objectFit: 'cover', borderRadius:
'8px' }} />
            <div>
              <h3>{item.producer}</h3>
              <p style={{ color: '#666' }}>Size: {item.size} | Color: {item.color}</p>
            </div>
          </div>

          {/* лічильник + ціна */}

```

```

    <div style={{ display: 'flex', alignItems: 'center', gap: '20px' }}>

      { /* КНОПКИ +/- */ }

      <div style={{ display: 'flex', alignItems: 'center', border: '1px solid #ddd',
borderRadius: '5px' }}>
        <button
          onClick={() => dispatch(removeOneFromCart(item.id))} // <-- Было
removeFromCart, стало removeOneFromCart
          style={{ background: 'none', border: 'none', padding: '5px 10px', cursor:
'pointer', fontSize: '16px' }}
        >
          -
        </button>
        <span style={{ padding: '0 10px', fontWeight: 'bold' }}>{item.count}</span>
        <button
          onClick={() => dispatch(addToCart(item))} // Додает еще один такой самый
          style={{ background: 'none', border: 'none', padding: '5px 10px', cursor:
'pointer', fontSize: '16px' }}
        >
          +
        </button>
      </div>

      <span style={{ fontWeight: 'bold', fontSize: '18px' }}>
        ${ (item.price * item.count).toFixed(2) }
      </span>
    </div>
  </div>
  )))
</div>

<div style={totalStyles}>
  Total amount: ${totalAmount.toFixed(2)}
</div>

<div style={{ marginTop: '30px', textAlign: 'right' }}>
  <Link to="/catalog"><PrimaryButton>Back to Catalog</PrimaryButton></Link>
</div>
</div>
);
}

export default CartPage;

```

## Файл actions.js:

```
export const ADD_TO_CART = 'ADD_TO_CART';
export const REMOVE_FROM_CART = 'REMOVE_FROM_CART';
export const REMOVE_ONE_FROM_CART = 'REMOVE_ONE_FROM_CART';

// Action Creators (функції, що створюють об'єкти дій)
export const addToCart = (item) => ({
  type: ADD_TO_CART,
  payload: item,
});

export const removeFromCart = (itemId) => ({
  type: REMOVE_FROM_CART,
  payload: itemId,
});

export const removeOneFromCart = (itemId) => ({
  type: REMOVE_ONE_FROM_CART,
  payload: itemId,
});
```

## Файл reducers.js:

```
import { ADD_TO_CART, REMOVE_FROM_CART, REMOVE_ONE_FROM_CART } from './actions';

const initialState = {
  cartItems: [],
};

const cartReducer = (state = initialState, action) => {
  switch (action.type) {

    case ADD_TO_CART:
      return {
        ...state,
        cartItems: [...state.cartItems, action.payload], //додавання в масив
      };

    case REMOVE_FROM_CART:
      return {
        ...state,
        //Залишається у новому списку тільки ті товари, ID яких не дорівнює тому, що ми хочемо видалити
        cartItems: state.cartItems.filter((item) => item.id !== action.payload),
      };

    case REMOVE_ONE_FROM_CART: {
```

```

    // Шукаємо ІНДЕКС (номер у списку) першого товару з таким ID
    const index = state.cartItems.findIndex(item => item.id === action.payload);

    if (index !== -1) {
        const newCartItems = [...state.cartItems];
        newCartItems.splice(index, 1);

        return {
            ...state,
            cartItems: newCartItems,
        };
    }
    return state;
}

default:
    return state;
}
};

export default cartReducer;

```

### Файл store.js:

```

import { createStore } from 'redux';
import cartReducer from './reducers';

// Створюємо сховище, передаючи йому наш ред'юсер, який буде казати сховищу, як змінювати дані
const store = createStore(cartReducer);

export default store;

```

### Посилання на лабораторну роботу на гітхабі:

[https://github.com/IrynaLutsan/Web\\_Programming\\_Labs/tree/main/Labs\\_React](https://github.com/IrynaLutsan/Web_Programming_Labs/tree/main/Labs_React)

### Висновок:

У цій роботі було інтегровано Redux для управління станом кошика. Було налаштовано архітектуру (Store, Actions, Reducer), реалізовано логіку оновлення даних з дотриманням принципу незмінності та використані хуки useSelector і useDispatch для зв'язку компонентів зі сховищем. Це дозволило організувати чіткий однонаправлений потік даних та уникнути зайвої передачі пропсів.